

Basic (your first working tool) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangChain/Python), **Advanced** (how a real tool-calling round trip actually finishes). Read Basic first even if you already know this — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You need a normal Python function, marked as a "tool" the model is allowed to use.

The `@tool` decorator does that marking. Under it, write a completely ordinary function — it does the actual math, nothing fancy.

Then, when you call the model, you tell it "here's a list of tools you're allowed to use" — and ask a question that clearly needs one of them.

The model won't compute the answer itself here — it will ask to *call* your function. Your job in this exercise is to see that request happen, print it, then actually run the function yourself and print the real result.

Things to use:

- `from langchain_core.tools import tool`
- `@tool` above your function, with a docstring describing what it does.
- `model_with_tools = ChatOpenAI().bind_tools([add])`
- `response = model_with_tools.invoke("What's 5 + 7?")`
- `print(response.tool_calls)`

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

`@tool` (from `langchain_core.tools`) wraps a normal typed function into a `Tool` object that carries its name, its docstring (used as the description), and its argument shape — all derived automatically from the function's own signature and type hints. This is why type hints on tool functions aren't optional the way they might be on a throwaway script: the model literally reads them.

The docstring matters just as much — `@tool` reads it as the tool's description shown to the model, so a missing or vague docstring is exactly the "vague description" problem the Core Concepts section warns about, even on this first, simple example.

Registering the tool on a model call — `model.bind_tools([add])` — returns a new model wrapped with that tool list attached. `bind_tools()` takes a list even for one tool, since it's the same shape you'll extend in later exercises with more tools. Calling `.invoke(...)` on it doesn't run your function; it returns an `AIMessage` whose `.tool_calls` attribute is a list of dictionaries, each with `name`, `args`, and an `id` — that's everything your code needs to actually go run the right function with the right arguments.

Actually running it: `add.invoke(response.tool_calls[0]["args"])` — note this uses `.invoke(...)` on the tool itself, not a plain Python function call, since `@tool` wraps it into a `Runnable` too.

Write out, in plain words, the two-step handoff: the model requests a call, and your code is the one that actually executes it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

"Run the tool and print the real result" is where most people stop — but it isn't actually the end of a real tool-calling round trip. Look back at the Core Concepts section's 5-step trace: step 4 is "send the function's result back to the model, labeled as belonging to that specific tool call," and step 5 is the model's *actual* final answer, using that result. Stopping at step 3 means you've proven the plumbing works, but you never see the model finish the job it started.

Sending the result back correctly means tagging it with `call["id"]`, not just handing back a bare string — that `id` is what lets the model match "here's the result" to "here's the request I made," which matters the moment more than one tool call is in flight at once.

That leads to the real edge case this exercise's tidy example hides: **what if `response.tool_calls` has more than one entry?** Models can request several tool calls in a single response (this is called parallel tool calls) — a question like "what's 5 + 7, and also what's 3 + 4?" could easily produce two `add` calls in one response, not one. Code that only ever looks at `response.tool_calls[0]` silently drops the second request.

The extra pieces needed for a real round trip:

- `from langchain_core.messages import ToolMessage` — the message type used to hand a tool's result back, tagged to the request it answers.
- `ToolMessage(tool_call_id=call["id"], content=str(result))` — built once per tool call, using that call's own `id`.
- A loop over **every** entry in `response.tool_calls`, not just the first — so a parallel-call response gets fully answered, not partially.
- A second `.invoke(...)` call, passing the full message history (the original question, the model's tool-call message, and every `ToolMessage`) back to the model — this is the call that actually produces the natural-language final answer.

Sketch this full round trip yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the pieces and stops once the tool has actually run once. Intermediate explains what each of those pieces does under the hood, and adds real type-hint/docstring awareness. Advanced goes past "run it once and print it" to the part the tidy example never forces you to confront — a response can carry more than one tool call at once, and a real round trip isn't finished until the result is handed back to the model, tagged to the exact request it answers, so the model can produce its actual final reply.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define a tool called add that takes two numbers and returns their sum

register it on a model

ask the model "what's 5 + 7?"

print what tool call the model requested (name, arguments)

actually run the tool with those arguments

print the real result
```

Here's almost the whole thing — just try running it and reading it line by line:

```
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b

model_with_tools = ChatOpenAI().bind_tools([add])
```

Expected output if you run just this (nothing calls the model yet): nothing — building `model_with_tools` doesn't ask it anything. Add `response = model_with_tools.invoke("What's 5 + 7?")` and `print(response.tool_calls)` below it to see the request print.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
@tool
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b

model_with_tools = ChatOpenAI(model="...", temperature=0).bind_tools([add])

response = model_with_tools.invoke("What's 5 + 7?")
print(response.tool_calls)

for call in response.tool_calls:
    if call["name"] == "add":
        result = add.invoke(call["args"])
        print(f"Real result: {result}")
```

What's missing from a real ship-it version: checking `response.tool_calls` isn't empty before looping (the model could just answer directly), and checking `call["name"]` matches what you expect instead of assuming it. Add both, then compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
define add as before, register it on a model

ask "what's 5 + 7? also what's 3 + 4?" (deliberately asks for two calls)

response = model_with_tools.invoke(question)

if no tool_calls: print the direct answer, stop

messages = [the original question, response itself]

for each call in response.tool_calls:
    run the tool for that call
    wrap the result in a ToolMessage tagged with call["id"]
    add that ToolMessage to messages

final_response = model_with_tools.invoke(messages)
print(final_response.content) # the model's real final answer
```

Here's almost the whole thing — fill in the missing piece yourself:

```
from langchain_core.messages import ToolMessage
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b

model_with_tools = ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([add])

question = "What's 5 + 7? Also, separately, what's 3 + 4?"
response = model_with_tools.invoke(question)
print(f"Requested {len(response.tool_calls)} tool call(s): {response.tool_calls}")

messages = [{"role": "user", "content": question}, response]

# your turn: loop over every call in response.tool_calls, run add.invoke(call["arguments"])
# for each one, build a ToolMessage(tool_call_id=call["id"], content=str(result))
```

```
# and append it to messages
...

final_response = model_with_tools.invoke(messages)
print(f"Model's final answer: {final_response.content}")
```

Fill in the loop yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same underlying idea (register, ask, run) at 3 completeness levels — Basic's version stops the instant it has the real result of one call, Intermediate adds the defensive checks a real ship-it version needs (still stopping at one call), and Advanced closes the loop the Core Concepts trace actually describes — handling every tool call in the response, not just the first, and sending each result back tagged to its own `id` so the model can produce a genuine final answer instead of you just reading the raw number off the screen yourself.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)